Meta Pre-Training for Few-Shot Post-Training of Robot Policies

Anonymous CVPR submission

Paper ID *****

Abstract

Robot foundation models, from vision-language-action models to world action models, are pre-trained on web-scale video and cross-embodiment robot data and then post-trained on demonstrations of a target robot and environment. This recipe works well when hundreds of demonstrations are available, but we show that it degrades sharply when only a few demonstrations per task can be collected: post-training the pre-trained Cosmos3-Edge model on three demonstrations per task reduces its LIBERO success rates from 87–99% to 58–91%. We argue that pre-training on a multi-embodiment mixture is not the same as learning to adapt *from few demonstrations, and propose Meta Pre-Training, a stage inserted between pre-training and post-training. Using robot datasets that are already part of the pre-training mixture, the stage repeatedly simulates few-shot post-training—a short fine-tuning on a few demonstrations of one source embodiment—and moves the weights toward the adapted solution with a first-order interpolation update. Because the update is a convex combination of weights, the entire trainable set of the post-training recipe can be meta-trained with the standard sharded training stack, and the result is a drop-in replacement for the pre-trained weights. We further observe that few-shot post-training over-relies on the visual context and under-uses the instruction, and introduce ranking policy learning, a post-training objective that samples negative instructions and requires the action prediction under a wrong instruction to be worse than under the correct one. With three demonstrations per task, our approach raises the average success rate over LIBERO-10, -Goal, -Spatial, and -Object from 71.7% to 78.7% (58.0/63.2/74.8/90.6% to 69.4/76.2/78.6/90.4%) and improves every LIBERO-10 task, whereas ordinary training on the same source data yields no gain (58.2% on LIBERO-10). The benefit is not limited to the few-shot regime: with all demonstrations, LIBERO-10 improves from 87.2% to 94.6%.*

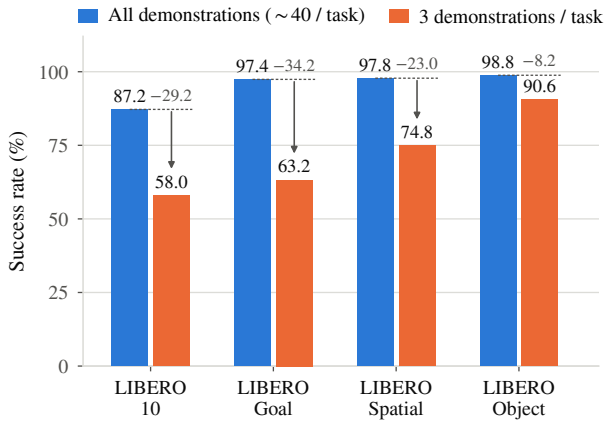


Figure 1. **Few-shot post-training collapses.** Closed-loop success rate of the pre-trained Cosmos3-Edge model after post-training on all demonstrations of each LIBERO suite (blue; 379–454 demonstrations, about 40 per task) and on three demonstrations per task (orange; 30 per suite), with the post-training recipe otherwise identical. The same recipe loses 8 to 34 points when demonstrations are scarce. Every bar is 50 trials on each of the 10 tasks of a suite.

1. Introduction

Progress in artificial intelligence over the past decade has followed a simple pattern: large models are pre-trained on broad data and then adapted to a target task from comparatively little data. Robot learning has adopted the same pattern. Vision-language-action models (VLAs) [3, 23, 35, 55] and, more recently, world action models (WAMs) that generate future video and robot actions jointly [7, 9, 36, 47, 54] are pre-trained on web-scale video and cross-embodiment robot data [22, 38], and are then *post-trained* on demonstrations collected with the target robot in the target environment, where an action objective turns the pre-trained network into a policy. When the post-training set is large, the recipe is remarkably effective. On the four LIBERO suites [29], post-training the pre-trained Cosmos3-Edge model [36] on all available demonstrations reaches 87–99% success (Fig. 1, blue).

Demonstrations, however, are the most expensive ingredient of this pipeline. Every new robot, workcell, or task re-

quires a person to teleoperate it, and in many deployments only a handful of demonstrations per task is affordable. In this regime the standard recipe breaks down. With very few demonstrations per task, the same post-training recipe loses between 8 and 34 points on the four LIBERO suites (Fig. 1, orange), and the loss is largest on the long-horizon and the goal-conditioned suites. A model that has been pre-trained on millions of robot and human videos does not turn a few demonstrations into a reliable policy.

We attribute this gap to a mismatch between what pre-training optimizes and what few-shot post-training needs. Pre-training optimizes the weights for the *joint* objective over a mixture of embodiments and tasks, and nothing in that objective asks the weights to be a good *starting point* for a short fine-tune on an embodiment outside the mixture. Being good at many embodiments and adapting effectively to a new one from few demonstrations are different properties, and a pre-trained model that has the first does not automatically have the second. A second, complementary effect appears during post-training itself. With only a few demonstrations, the policy latches onto the visual context of each demonstration, which already identifies the task in most benchmarks, and largely ignores the language instruction; in scenes that are shared by several tasks it then executes the wrong task.

We address the first problem with Meta Pre-Training, a stage inserted between the existing pre-training and post-training (Fig. 2). The stage builds episodes from five robot datasets of different embodiments, each episode simulating the downstream problem: a few demonstrations of one source embodiment and a short fine-tuning on them. After each episode, the weights are moved a fraction of the way toward the adapted solution, following first-order meta-learning [33]. Because this update is a plain interpolation in weight space, the entire trainable set of the post-training recipe can be meta-trained with the standard training stack, and the resulting checkpoint is a drop-in replacement for the pre-trained weights: post-training proceeds exactly as before, only from a better starting point. We address the second problem with Ranking Policy Learning (RPL), a post-training objective that samples negative instructions and requires the action prediction under a wrong instruction to be worse than under the correct one, which forces the policy to commit to the instructed behavior rather than to the behavior suggested by the scene.

On the four LIBERO suites with three demonstrations per task, our approach raises the average success rate from 71.7% to 78.7%, with gains of 11.4 points on LIBERO-10 and 13.0 points on LIBERO-Goal, and improves every one of the ten LIBERO-10 tasks. Ordinary training on the same source datasets yields no gain (58.0% to 58.2% on LIBERO-10), so the improvement comes from meta-learning rather than from additional data. The meta pre-trained initialization also helps when data is plentiful: with all LIBERO-10 demonstrations the success rate rises from 87.2% to 94.6%. In summary, our contributions are:

- We identify and quantify the few-shot post-training problem of pre-trained robot policies on the four LIBERO suites, where the standard recipe loses 8–34 points when only a few demonstrations per task are available.
- We propose Meta Pre-Training, a stage between pre-training and post-training that optimizes the entire trainable set of the post-training recipe for few-shot adaptation while leaving the architecture and the post-training recipe unchanged, and Ranking Policy Learning, a post-training objective that counteracts the visual bias of few-shot policies.
- We show consistent gains in the few-shot regime, a control experiment in which ordinary training on the same source data gives none, and a gain in the full-data regime.

2. Related Work

2.1. Few-Shot Learning

Meta-learning. Few-shot learning asks a model to solve a new task from a few examples, and meta-learning makes this ability the training objective [17]. Metric-based methods compare queries to a support set in a learned embedding space [43, 44]. Optimization-based methods instead learn an initialization from which a few gradient steps suffice. MAML [11] differentiates through the inner loop, its first-order variant drops the second-order terms, and Reptile [33] replaces the meta-gradient by an interpolation toward the weights reached after k inner steps, which decouples the inner horizon from the memory footprint. ANIL [41] shows that much of MAML's benefit on few-shot classification comes from feature reuse with a rapidly adapting head, and [1] documents the instability of MAML-style training. Our meta pre-training adopts the interpolation update of [33], but applies it to the full post-training recipe of a billion-parameter policy backbone rather than to a few steps on a small network, meta-learns the action head of an unseen embodiment through a shared head, and relies on the fact that the update is a convex combination of weights, which lets the standard sharded training stack run the inner loop.

Few-shot imitation and data-efficient fine-tuning. In robotics, one-shot imitation learning conditions a policy on a demonstration [10, 20], and meta-imitation learning applies MAML to visuomotor policies so that a single demonstration adapts them [12, 50]; skill-based methods adapt a pre-trained skill prior to a few demonstrations [15]. These works meta-train small policies on tens of tasks of one robot. A complementary line improves data efficiency through pre-trained visual representations [31, 32] or by fine-tuning generalist policies [3, 23, 37, 39, 42], typically with parameter-efficient adapters [18, 24]. Their fine-tuning

sets still contain tens to hundreds of demonstrations per task; the regime of a handful of demonstrations per task with a billion-parameter model has, to our knowledge, not been studied. Our method is orthogonal to the choice of adapter and recipe: it changes the point from which post-training starts and adds an objective to post-training, not the model.

Language grounding in policies. Language-conditioned policies are known to under-use the instruction when the scene alone predicts the behavior, a failure that the goal-conditioned LIBERO suite was designed to expose [29]. Classifier-free guidance [16] strengthens the conditioning at inference time, and contrastive objectives [40] align modalities with negatives at training time. Our ranking objective brings negatives into policy post-training: it ranks the action predictions under the correct and under an incorrect instruction, so that the policy cannot fit the demonstrations from the visual context alone.

2.2. World Action Models

World models learn to predict the consequences of actions [13, 14]. With video generation this idea has scaled to pixels. UniPi [9] plans by generating a video and recovers actions with an inverse dynamics model; GR-1 and GR-2 [8, 47] pre-train a video generator and then predict actions alongside future frames; VPP [19] feeds the representations of a video diffusion model to a policy; Genie [5] and LAPA [49] learn latent actions from unlabeled videos; DreamGen [21] uses a post-trained video world model to synthesize training trajectories; and V-JEPA 2 [2] plans in a learned latent space. A growing family couples video and action generation in one network, which we refer to as world action models: UWM [54] couples video and action diffusion, WorldVLA [7] autoregresses over both modalities, Genie Envisioner [27] builds a policy on a video world model, and Cosmos [34, 36] unifies language, video, and action generation in a mixture-of-transformers backbone [26, 52]. Vision-language-action models [3, 4, 23, 35, 37, 55] are the action-only relatives of these models, trained on cross-embodiment collections [22, 38]. All of these models are post-trained on target data with tens to hundreds of demonstrations per task; we study how to make this post-training work with a handful. We use Cosmos3-Edge [36] as our backbone, but our method only assumes a pre-trained policy backbone with an action objective, a post-training recipe, and a set of source robot datasets; it does not use the video generation ability of the backbone and applies to VLAs in the same way.

3. Method

We describe our method with Cosmos3-Edge [36] as the representative backbone. The method only assumes a pre-trained policy backbone trained with an action objective, a post-training recipe, and a set of source robot datasets, and applies to VLAs in the same way. We first describe the backbone and its post-training recipe (Sec. 3.1), then define the few-shot post-training problem (Sec. 3.2), present Meta Pre-Training (Sec. 3.3) and Ranking Policy Learning (Sec. 3.4), and finally summarize how post-training uses them (Sec. 3.5). Fig. 2 gives an overview.

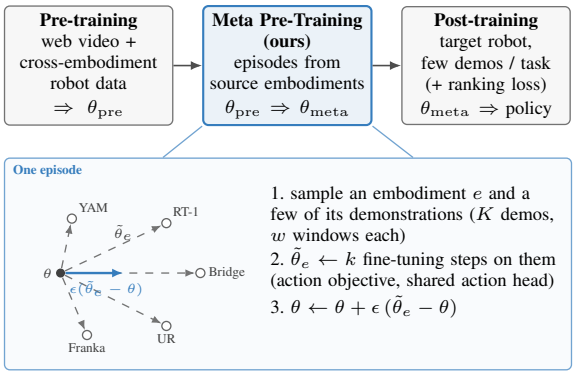


Figure 2. **Overview.** We insert a Meta Pre-Training stage between pre-training and post-training. Each episode simulates few-shot post-training on one source embodiment (a few demonstrations, k fine-tuning steps; dashed arrows to the fine-tuned weights $\tilde{\theta}_e$) and moves the weights a fraction ϵ toward the adapted solution (blue arrow). The result θ_{meta} is loaded by the post-training recipe exactly like the pre-trained checkpoint; post-training can additionally use the ranking loss of Sec. 3.4.

3.1. Preliminaries: Backbone and Post-Training

Model. Cosmos3-Edge [36] is a mixture-of-transformers model [26] with two towers coupled by joint attention: a *reasoner* tower that processes the text instruction and the conditioning tokens autoregressively, and a *generator* tower that denoises video latents and action tokens with a rectified-flow objective [28, 30]. Video is tokenized by a causal spatio-temporal VAE [46]. Actions of every embodiment are expressed in a unified representation: the relative end-effector translation, a 6-D rotation [53], and a gripper state per arm, zero-padded to a fixed width. They enter and leave the token stream through embodiment-specific *action heads*. A head consists of an input projection, an output projection, and a modality embedding, with one row per embodiment. When used as a policy, the model conditions on the current observation and the instruction and jointly generates the following video frames and the corresponding chunk of actions. Our method uses only the action pathway of this backbone; the video pathway is kept as it is in the recipe we inherit.

Objective. For a dataset $\mathcal{D}$ of windows, each consisting of an observation, an instruction, the following video frames,

and the corresponding actions, the training loss is

$$\mathcal{L}(\theta;\mathcal{D}) = \lambda_v\,\mathcal{L}_{\text{video}}(\theta;\mathcal{D}) + \lambda_a\,\mathcal{L}_{\text{action}}(\theta;\mathcal{D}), \quad (1)$$

two flow-matching losses on the video latents and the action tokens, weighted by λ_v and λ_a. For a VLA, $\mathcal{L}$ reduces to the action term.

Post-training recipe. Given a target dataset $\mathcal{D}_\mathcal{T}$, post-training minimizes Eq. (1) over a trainable subset θ of the weights, the generator tower together with its timestep embedding, its latent projections, and the action head of the target embodiment, while the reasoner tower and the tokenizers stay frozen. The target embodiment is new, so its action-head row is initialized randomly. We write $\mathrm{PT}_T(\theta_0;\mathcal{D}_\mathcal{T})$ for the weights after T optimizer steps of this recipe from the initialization θ_0. The standard pipeline uses the pre-trained weights, $\theta_0 = \theta_{\text{pre}}$.

3.2. Few-Shot Post-Training

We consider the regime in which $\mathcal{D}_\mathcal{T}$ holds only n demonstrations per task, with n small. We deliberately keep the recipe fixed, so that the only free variable is the initialization θ_0, and study which θ_0 maximizes the closed-loop performance of $\mathrm{PT}_T(\theta_0;\mathcal{D}_\mathcal{T})$ when $\mathcal{D}_\mathcal{T}$ is small. Fig. 1 shows that θ_{pre} is a poor choice. Pre-training minimizes $\mathbb{E}_e[\mathcal{L}(\theta;\mathcal{D}_e)]$ over a mixture of embodiments e, *i.e.*, it looks for one set of weights that is good for all of them at once. Few-shot post-training needs something else: weights from which a short optimization on a *new* embodiment lands in a good region. The two coincide only if the per-embodiment solutions lie close to the joint solution. We hypothesize that they do not, and that this mismatch, together with the randomly initialized action head, is what makes post-training from θ_{pre} data hungry.

3.3. Meta Pre-Training

We therefore add a stage that directly optimizes θ for being a good starting point of PT on an unseen embodiment, using source embodiments as stand-ins for the target. The procedure is summarized in Alg. 1.

Source episodes. We use E robot datasets that are already part of the pre-training mixture, each recorded with a different embodiment (Sec. 4.1 lists them); no new data is collected. Every source is read through the same windowing and transformation pipeline as the target, so meta-training samples have exactly the format that post-training samples have. An episode samples an embodiment $e \sim \mathrm{Unif}\{1,\dots,E\}$ and K demonstrations of e, and takes w windows from each of them. We call the resulting set of Kw windows the *support set* $\mathcal{S}_e$; it is the only data that the inner loop of the episode sees. Sampling whole demonstrations rather than windows matters: the downstream data consists of a few complete trajectories, and the support set should have the same structure, a few trajectories with many windows each.

Algorithm 1 Meta Pre-Training

Require: pre-trained weights θ_{pre}; source datasets $\{\mathcal{D}_e\}_{e=1}^{E}$; fine-tuning step FT; K, w, k, N; schedule $\epsilon_1 \geq \cdots \geq \epsilon_N$

$\theta \leftarrow \theta_{\text{pre}}$; initialize the shared action head randomly
for $i = 1,\dots,N$ **do**
  sample $e \sim \mathrm{Unif}\{1,\dots,E\}$ and K demonstrations of $\mathcal{D}_e$
  $\mathcal{S}_e \leftarrow w$ windows from each demonstration
  $\tilde{\theta}_e \leftarrow \mathrm{FT}_k(\theta;\mathcal{S}_e)$ ▷ k fine-tuning steps on $\mathcal{S}_e$, fresh optimizer state
  $\theta \leftarrow \theta + \epsilon_i\,(\tilde{\theta}_e - \theta)$
end for
return $\theta_{\text{meta}} = \theta$ ▷ trainable set and shared action head

Shared action head. The target embodiment arrives without an action head. To meta-learn the head as well, all source embodiments share one spare action-head row during meta pre-training, initialized from scratch like the downstream head. At post-training time this row is copied into the row of the target embodiment, so the target head starts from a meta-learned initialization instead of a random one.

Inner loop. From the current weights θ, the inner loop runs k optimizer steps of ordinary fine-tuning on the support set $\mathcal{S}_e$, *i.e.*, on the windows sampled from the K demonstrations of embodiment e, with the objective of Eq. (1); the optimizer state is reset at every episode. We write $\tilde{\theta}_e = \mathrm{FT}_k(\theta;\mathcal{S}_e)$ for the fine-tuned weights. Each inner step covers the whole support set, so the k steps make k passes over the K demonstrations, a miniature of post-training, which makes many passes over a few demonstrations.

Outer update. The outer update interpolates between the current and the adapted weights,

$$\theta \leftarrow \theta + \epsilon\,(\tilde{\theta}_e - \theta), \quad (2)$$

with ϵ annealed over the N episodes. This is the first-order update of [33]. In expectation, Eq. (2) converges to the point at which k steps of fine-tuning on a randomly drawn source embodiment no longer move the weights on average. That point lies at the center of the per-embodiment k-step solutions: a few fine-tuning steps suffice to reach the solution of every source embodiment. The source embodiments are stand-ins for the target. The target embodiment is never seen during meta pre-training, but the property being optimized, that a few demonstrations and a few steps are enough to adapt, is enforced across embodiments that differ in robot, cameras, and action space, and we ex-

pect it to carry over to a new embodiment in the same way that a meta-learned initialization transfers to unseen tasks in few-shot learning [11, 33]; Sec. 4 confirms this empirically. Nichol *et al*. [33] further show that for $k > 1$ the expected update contains, besides the average gradient, a term that increases the inner product between the gradients of different minibatches of the same task. In our setting that term rewards weights for which fitting a few windows of an embodiment also reduces the loss on its remaining windows, which is precisely the generalization that few-shot post-training requires.

Why an interpolation update. Three properties make the interpolation update the right tool at this scale. *(i) Long inner horizon.* The downstream adaptation runs for thousands of optimizer steps. Meta-gradient methods such as first-order MAML, whose update is the query-set gradient at the adapted point, are tied to the short inner horizon they are trained with; in preliminary experiments with a short inner loop they produced initializations whose offset from θ_{pre} the subsequent post-training neither used nor undid. The interpolation update has no query set, and its k is limited only by compute. *(ii) Memory.* Eq. (2) needs one extra copy of the trainable parameters and nothing else; the model's own sharded parameters [51] are the fast weights, and the standard trainer, optimizer, activation checkpointing, and checkpoint code run the inner loop unchanged. This is what makes meta-training the entire trainable set affordable on a few GPUs, whereas an inner loop with per-step fast weights or a query gradient at the adapted point would not fit. *(iii) Compatibility.* The outer update is a convex combination of weights, so θ remains a valid checkpoint of the unchanged architecture at every iteration and is loaded by the post-training recipe without modification. Meta-training the whole trainable set rather than only the heads is important: in preliminary experiments, meta-learning only the action heads (an ANIL-style variant [41]) gave at most marginal gains, because full-parameter post-training then overwrites the small meta-learned subset.

3.4. Ranking Policy Learning

Visual bias of few-shot policies. A good initialization does not remove a second difficulty of few-shot post-training. With n demonstrations per task, the training set contains a few trajectories per instruction, and in each of them the visual context, the objects and their layout at the start of the trajectory, is almost perfectly predictive of the behavior that follows. The policy can therefore fit the demonstrations without reading the instruction, and the objective of Eq. (1) provides no signal against this shortcut: it only asks the prediction under the correct instruction to be accurate. At test time in a scene that is shared by several tasks, such a policy executes the behavior that the scene suggests rather than the one that was instructed, and in our rollouts this is the dominant failure mode on the suites whose tasks share one scene (Sec. 4.5). We refer to this effect as *visual bias*.

Objective. Ranking Policy Learning (RPL) adds a signal that depends on the instruction. For each training window with observation o, instruction c, and target actions, we sample a negative instruction c^- from the other tasks of the target set and evaluate the action flow-matching error under both instructions, with the same noised actions x_t, the same timestep t, and the same observation:

$$E^+ = \left\| v_\theta(x_t, t, o, c) - v^\star \right\|^2, \qquad E^- = \left\| v_\theta(x_t, t, o, c^-) - v^\star \right\|^2, \tag{3}$$

where v_θ is the predicted and $v^\star$ the target velocity of the action tokens. The ranking loss

$$\mathcal{L}_{\text{rank}} = \mathbb{E}\big[\max\big(0,\, m + E^+ - E^-\big)\big] \tag{4}$$

requires the prediction under the wrong instruction to be worse than under the correct one by a margin m, and the post-training objective becomes $\mathcal{L} + \lambda_r \mathcal{L}_{\text{rank}}$. The hinge is inactive once the margin is met, so windows whose actions already depend on the instruction are left alone, and it pushes the two predictions apart otherwise, so that the instruction, not only the scene, determines the action. The negative branch decodes actions only and adds no video objective under the wrong instruction; windows whose instruction is dropped for classifier-free training contribute no ranking loss. RPL adds no parameters and no data, and costs one additional forward and backward pass of the generator per window.

3.5. Post-Training with Meta Pre-Trained Weights

Post-training from θ_{meta} differs from the baseline in two places: the trainable set is loaded from θ_{meta} instead of θ_{pre}, and the target action-head row is initialized from the shared meta-learned head. Where RPL is used, the ranking loss of Eq. (4) is added to the objective. The frozen reasoner tower and tokenizers, the data pipeline, the optimizer, the schedule, the number of steps, and the inference procedure are unchanged. The same θ_{meta} is used for every target suite and every data budget in Sec. 4; meta pre-training is a one-time cost that is amortized over all downstream targets.

4. Experiments

4.1. Setup

Benchmark. We evaluate on the four LIBERO suites [29], each with 10 language-conditioned manipulation tasks: LIBERO-Spatial (same objects, different layouts), LIBERO-Object (different objects, same layout), LIBERO-Goal (same scene and objects, different goals), and LIBERO-10 (long-horizon, two-stage tasks). We use the 20 Hz LeRobot release of the demonstrations [6], which

contains 379, 428, 432, and 454 successful demonstrations for LIBERO-10, -Goal, -Spatial, and -Object.

Few-shot protocol. Our main experiments use n=3 demonstrations per task. For every suite we select them with a fixed random seed, 30 demonstrations in total, and keep this selection fixed for all methods. Five further demonstrations per task, disjoint from the training set, are held out to monitor the validation loss; they are never used for training or for checkpoint selection, and we always evaluate the final checkpoint of each run.

Models. **PT** (Cosmos3-Edge-PT) post-trains the pre-trained Cosmos3-Edge checkpoint with the standard recipe; this is the baseline of Fig. 1. **META-PT** (Cosmos3-Edge-META-PT) post-trains from our meta pre-trained checkpoint θ_{meta} as described in Sec. 3.5; on LIBERO-Goal and LIBERO-Spatial, whose ten tasks share one scene, post-training additionally uses RPL (Sec. 3.4). **JOINT-PT** is the control of Sec. 4.3: it post-trains from weights that were trained on the same source episodes as θ_{meta} by ordinary joint training instead of meta pre-training. All runs of a suite share every setting, including the training subset and the seed, and differ only in the initialization, the action-head learning rate noted below, and the ranking loss where stated.

Implementation details. The backbone is the pre-trained Cosmos3-Edge checkpoint [36]: two towers of 28 layers with hidden size 2048, of which the post-training recipe trains the generator tower, its timestep embedding, its latent projections, and the action head, 1.42B parameters in total. A window consists of the current observation, the next 16 video frames at 20 Hz, and the corresponding 16 actions; the policy observes the third-person and the wrist camera at 256×256, concatenated side by side, and predicts 10-D actions (relative end-effector translation, 6-D rotation, and gripper), normalized with the quantile statistics of the suite. Post-training uses $\lambda_v = \lambda_a = 10$, AdamW with learning rate 5×10^{-5}, weight decay 0.05, $\beta = (0.9, 0.99)$, 200 warm-up steps and linear decay to zero at T=2000 steps, batches of 256 windows, an exponential moving average of the weights, bf16 compute with fp32 master weights, and FSDP over two 48 GB GPUs; the instruction is dropped for 10% of the samples. The randomly initialized action head of PT is trained with a 5× learning-rate multiplier, as in the recipe of [36]; the meta-learned head of META-PT and JOINT-PT uses the base learning rate. Meta Pre-Training uses E=5 source datasets of different embodiments that are part of the pre-training mixture: RT-1 (Google Robot) [4], BridgeData V2 (WidowX) [45], the UR-5e and the Franka subsets of RoboMIND [48], and the bimanual YAM data of MolmoAct [25], whose 20-D actions exercise the padded action representation. We run N=1000 episodes with K=32 demonstrations and w=8 windows per demonstration (256 windows per episode) and k=10 inner steps. The inner optimizer is AdamW with the learning rate and weight decay of the post-training recipe, so that the inner loop resembles the downstream adaptation, with two warm-up steps, gradient clipping at 1.0, and a fresh optimizer state per episode; ϵ decays linearly from 0.5 to 0.05. The run takes 1.5–2 minutes per episode on four 48 GB GPUs, 25–30 hours in total, with a peak memory below 30 GB per GPU, and consumes about 32k demonstration samples, a negligible fraction of the pre-training data; the final θ_{meta} is used for all experiments. RPL uses margin m=0.1 and weight λ_r=1, with one negative instruction per window drawn uniformly from the other nine tasks of the suite. JOINT-PT trains the same trainable set and shared action head on the same kind of source episodes (one embodiment per episode, 256 windows from a few of its demonstrations) for the same number of episodes, with the same optimizer, learning rate, and loss, but by ordinary joint training: each episode contributes one optimizer step and the weights are simply carried over, *i.e.*, k=1 and ϵ=1 in Alg. 1. The resulting checkpoint is then post-trained exactly like META-PT.

Model	LIBERO-10	Goal	Spatial	Object	Avg.
PT, all demonstrations	87.2	97.4	97.8	98.8	95.3
PT (baseline)	58.0	63.2	74.8	**90.6**	71.7
META-PT (ours)	**69.4**	**76.2**	**78.6**	90.4	**78.7**
Δ	+11.4	+13.0	+3.8	−0.2	+7.0

Table 1. **Few-shot post-training on LIBERO (3 demonstrations per task).** Success rate (%) over 50 trials on each of the 10 tasks of a suite. PT is Cosmos3-Edge-PT, post-trained from the pre-trained weights; META-PT is Cosmos3-Edge-META-PT, post-trained from our meta pre-trained weights (with RPL on Goal and Spatial). Both are post-trained with the identical recipe on the identical 30 demonstrations. The gray row is the baseline recipe trained on all demonstrations of the suite, for reference.

Evaluation. We run the policy closed loop in the LIBERO simulator, 50 trials per task with the benchmark's fixed initial states, *i.e.*, 500 episodes per suite and per model. Each predicted 16-step action chunk is executed before the policy is queried again; actions are sampled with 30 denoising steps and without classifier-free guidance. Episodes time out after 520, 300, 220, and 280 control steps for LIBERO-10, -Goal, -Spatial, and -Object. We report the success rate in percent. With 500 Bernoulli trials the standard error at a success rate of 70% is about 2.0 points, so differences of fewer than about four points should be read as ties.

4.2. Few-Shot Post-Training

Tab. 1 reports the main result. Starting post-training from θ_{meta} instead of θ_{pre} raises the average success rate over the four suites from 71.7% to 78.7%. The gains are largest where the baseline is weakest: +11.4 points on LIBERO-10 and +13.0 points on LIBERO-Goal, five to six stan-

#	Task (abbreviated instruction)	PT	META-PT	Δ
1	alphabet soup & tomato sauce in basket	20	**42**	+22
2	cream cheese & butter in basket	76	**80**	+4
3	turn on stove, moka pot on it	50	**60**	+10
4	black bowl in bottom drawer, close it	56	**66**	+10
5	white mug on left plate, other on right	50	**62**	+12
6	book in back compartment of caddy	74	**78**	+4
7	white mug on plate, pudding to its right	54	**74**	+20
8	alphabet soup & cream cheese in basket	58	**70**	+12
9	both moka pots on the stove	60	**78**	+18
10	yellow-white mug in microwave, close it	82	**84**	+2
	All tasks	58.0	**69.4**	+11.4

Table 2. **Per-task success rate (%) on LIBERO-10 with 3 demonstrations per task**, 50 trials per task. PT and META-PT denote Cosmos3-Edge-PT and Cosmos3-Edge-META-PT. The meta pre-trained initialization improves every task.

Model	Stage on the source episodes	LIBERO-10
PT	none ($\theta_0 = \theta_{\text{pre}}$)	58.0
JOINT-PT	ordinary joint training (k=1, ϵ=1)	58.2
META-PT	Meta Pre-Training (ours)	**69.4**

Table 3. **Meta-learning versus ordinary training on the same source data.** Success rate (%) on LIBERO-10 with 3 demonstrations per task, 500 episodes. JOINT-PT and META-PT consume the same source episodes with the same trainable set, optimizer, loss, and shared action head, and are post-trained with the identical recipe; only the update rule differs.

dard errors each, and +3.8 points on LIBERO-Spatial. On LIBERO-Object the two models tie (90.6% versus 90.4%); this suite is the easiest one, its objects differ in appearance so that the visual context resolves the task, and the baseline already comes within eight points of the full-data result from 30 demonstrations. Averaged over the suites, our approach closes 30% of the gap between few-shot post-training (71.7%) and post-training on all demonstrations (95.3%) without touching the model or the data, and the remaining gap shows that few-shot post-training is far from solved.

Per-task behavior. Tab. 2 breaks LIBERO-10 down by task. The improvement is not concentrated in a few tasks: all ten tasks improve, by 2 to 22 points. The largest gains appear on tasks with two sequential pick-and-place segments that involve visually similar containers (tasks 1, 7, and 9), where the baseline tends to complete only the first segment within the time limit, while the tasks on which the baseline is already strong (tasks 2, 6, and 10) change little.

4.3. Meta-Learning versus Ordinary Training on the Source Data

θ_{meta} has seen source demonstrations that θ_{pre} has not seen in this form, so the gains of Tab. 1 could in principle come from this additional exposure rather than from the meta-learning objective. Tab. 3 separates the two. JOINT-PT consumes the same source episodes with the same trainable set, optimizer, loss, and shared action head, but updates the weights by ordinary joint multi-embodiment training, one optimizer step per episode, instead of adapting for k steps and interpolating. Post-training from this checkpoint reaches 58.2% on LIBERO-10, indistinguishable from the 58.0% of the baseline, whereas post-training from θ_{meta} reaches 69.4%. Ordinary training on the source data therefore does not improve the starting point for few-shot post-training, even though it fits the same data with the same model; the improvement comes from optimizing the weights for adaptation, which is what Meta Pre-Training does. This is also the sense in which the stage uses no new data: the same episodes, consumed in the ordinary way, give nothing.

Training demonstrations	PT	META-PT	Δ
3 per task (30)	58.0	**69.4**	+11.4
all (379)	87.2	**94.6**	+7.4

Table 4. **Beyond the few-shot regime: LIBERO-10 with all demonstrations.** Success rate (%) over 500 episodes; PT and META-PT as in Tab. 1. Post-training from the meta pre-trained weights also improves the full-data setting, removing more than half of the remaining failures.

4.4. Beyond the Few-Shot Regime

The meta pre-trained initialization could act only as a regularizer for tiny datasets, or it could be a better starting point in general. Tab. 4 repeats the LIBERO-10 experiment with all 379 demonstrations and the same 2000-step schedule. Post-training from θ_{meta} raises the success rate from 87.2% to 94.6%, *i.e.*, the failure rate drops from 12.8% to 5.4%. The gain is smaller than in the few-shot regime, as expected when the target data can itself move the weights far, but it is well outside the evaluation noise. We conclude that θ_{meta} is a better initialization for post-training in general, and that its advantage grows as the target data shrinks.

4.5. Discussion and Limitations

What the gains do and do not show. The models of Tab. 1 share the training subset, the recipe, the seed, and the evaluation episodes, so the comparisons are paired, but each number is a single training run evaluated on 500 episodes; we therefore do not interpret the sub-four-point differences on LIBERO-Spatial and LIBERO-Object. The control of Tab. 3 matches the data and the number of episodes of meta pre-training but not the number of optimizer steps, since each episode contributes one step instead of k; a step-matched control is a natural addition.

Where the remaining failures are. Inspecting the rollouts of the few-shot policies on LIBERO-Goal and LIBERO-Spatial, whose ten tasks share one scene, we find that most failures are not imprecise motion but the execution of a different task of the same scene, or the manipulation of the

wrong instance of a repeated object. This is the visual bias that motivates RPL, and it explains why the ranking loss is most useful on these suites. On the long-horizon and the object suites, many windows are segments whose motion does not depend on the instruction, such as transporting or releasing an object; the absolute margin of Eq. (4) cannot be met on such windows, and in preliminary runs the ranking loss did not help there. A margin that adapts to the window, or negatives drawn from the tasks that the policy actually confuses, are natural refinements. Meta-training episodes that contain several instructions of the same scene are another direction, since our source episodes exercise the adaptation of dynamics and embodiment but not the grounding of several instructions in one scene.

Scope. We study one backbone and one model size, one meta pre-training budget, and simulated evaluation; the method itself only assumes a post-training recipe and a set of source robot datasets and should transfer to other WAMs and VLAs.

5. Conclusion

We showed that the standard post-training recipe of a pre-trained robot policy breaks down when only a few demonstrations per task are available, losing 8 to 34 points on the four LIBERO suites, and traced the problem to a mismatch between the objective of pre-training, which seeks weights that are good for a mixture of embodiments, and the need of few-shot post-training, which requires weights from which a short fine-tune on a new embodiment succeeds. Meta Pre-Training closes part of this gap with a stage that repeatedly simulates few-shot post-training on source embodiments already present in pre-training and moves the entire trainable set of the recipe toward the adapted solutions, and Ranking Policy Learning counteracts the visual bias of few-shot policies by ranking the action predictions under correct and incorrect instructions. The stage needs no new data, runs on the unchanged sharded training stack, and produces a checkpoint that the unchanged post-training recipe consumes. It improves the average success rate over the four suites in the few-shot regime, every task of LIBERO-10, and the full-data setting as well, while ordinary training on the same source data gives no gain. Margins and negatives that adapt to the window, meta-training with multi-instruction episodes to address instruction grounding, and scaling the stage to larger backbones and to VLAs are the next steps.

References

[1] Antreas Antoniou, Harrison Edwards, and Amos Storkey. How to train your MAML. In *International Conference on Learning Representations (ICLR)*, 2019.

[2] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023.

[5] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024.

[6] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. https://github.com/huggingface/lerobot, 2024.

[7] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21734*, 2025.

[8] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, Hanbo Zhang, and Minzhao Zhu. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024.

[9] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] Yan Duan, Marcin Andrychowicz, Bradly Stadie, Jonathan Ho, Jonas Schneider, Ilya Sutskever, Pieter Abbeel, and Wojciech Zaremba. One-shot imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[11] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*, 2017.

[12] Chelsea Finn, Tianhe Yu, Tianhao Zhang, Pieter Abbeel, and Sergey Levine. One-shot visual imitation learning via meta-learning. In *Conference on Robot Learning (CoRL)*, 2017.

[13] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[14] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by la-

tent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

[15] Kourosh Hakhamaneshi, Ruihan Zhao, Pieter Abbeel, Sergey Levine, and Ofir Nachum. Hierarchical few-shot imitation with skill transition models. In *International Conference on Learning Representations (ICLR)*, 2022.

[16] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[17] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 44(9):5149–5169, 2022.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[19] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning (ICML)*, 2025.

[20] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn. BC-Z: Zero-shot task generalization with robotic imitation learning. In *Conference on Robot Learning (CoRL)*, 2021.

[21] Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, et al. DreamGen: Unlocking generalization in robot learning through neural trajectories. *arXiv preprint arXiv:2505.12705*, 2025.

[22] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024.

[23] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.

[24] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[25] Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Wang, Sangho Lee, et al. MolmoAct: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.

[26] Weixin Liang, Lili Yu, Liang Luo, Srinivasan Iyer, Ning Dong, Chunting Zhou, Gargi Ghosh, Mike Lewis, Wen-tau Yih, Luke Zettlemoyer, and Xi Victoria Lin. Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models. *arXiv preprint arXiv:2411.04996*, 2024.

[27] Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, Liliang Chen, Shuicheng Li, Yang Wang, Shuangjie Chen, et al. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025.

[28] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[29] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[30] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[31] Arjun Majumdar, Karmesh Yadav, Sergio Arnaud, Yecheng Jason Ma, Claire Chen, Sneha Silwal, Aryan Jain, Vincent-Pierre Berges, Pieter Abbeel, Jitendra Malik, Dhruv Batra, Yixin Lin, Oleksandr Maksymets, Aravind Rajeswaran, and Franziska Meier. Where are we in the search for an artificial visual cortex for embodied intelligence? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[32] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2022.

[33] Alex Nichol, Joshua Achiam, and John Schulman. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*, 2018.

[34] NVIDIA. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025.

[35] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[36] NVIDIA. Cosmos 3: Omnimodal world models for physical AI. Technical report, NVIDIA, 2026. Technical report, https://research.nvidia.com/labs/cosmos-lab/cosmos3/.

[37] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.

[38] Open X-Embodiment Collaboration. Open X-embodiment: Robotic learning datasets and RT-X models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[39] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[40] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[41] Aniruddh Raghu, Maithra Raghu, Samy Bengio, and Oriol Vinyals. Rapid learning or feature reuse? Towards understanding the effectiveness of MAML. In *International Conference on Learning Representations (ICLR)*, 2020.

[42] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[43] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[44] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Koray Kavukcuoglu, and Daan Wierstra. Matching networks for one shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[45] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Conference on Robot Learning (CoRL)*, 2023.

[46] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[47] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2024.

[48] Kun Wu, Chengkai Zhu, Jiaming Hou, Zhengping Li, Ningbo Liu, Zhongyang Zhang, Yanjiang Cheng, et al. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024.

[49] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejune Joo, Jianwei Lin, Jiyoung Ahn, Linxi Fan, Yuke Kim, et al. Latent action pretraining from videos. In *International Conference on Learning Representations (ICLR)*, 2025.

[50] Tianhe Yu, Chelsea Finn, Annie Xie, Sudeep Dasari, Tianhao Zhang, Pieter Abbeel, and Sergey Levine. One-shot imitation from observing humans via domain-adaptive meta-learning. In *Robotics: Science and Systems (RSS)*, 2018.

[51] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.

[52] Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. *arXiv preprint arXiv:2408.11039*, 2024.

[53] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[54] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Robotics: Science and Systems (RSS)*, 2025.

[55] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.